

Day 41: Convolutional Layers (Beginner Edition)

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 6, Day 6

Every network from Day 36 through Day 40 treated its input as a flat vector — even Day 40's 2D signal-propagation demo flattened structure away before a single weight touched it. Real images lose meaning when flattened: a pixel's neighbors matter, and a 1000-pixel flat vector has no idea which pixels used to sit next to each other. Today builds a convolutional layer entirely from scratch — a small kernel slid across an image, computing one dot product per position — and trains a real (tiny) ConvNet that a flat dense layer would have a much harder time solving as directly.

0. Where Today Fits: From Flat Vectors to Spatial Structure

Start here (no prior knowledge assumed): Every classifier built through Day 40 fed its input straight into a matrix multiply — $X @ W$ — treating every input dimension as independent and interchangeable. An image is different: pixel (3,3) is meaningfully NEAR pixel (3,4) and far from pixel (0,0), and a flat vector representation throws that neighbor information away before the network ever sees it. Convolution is the layer type built specifically to preserve and exploit that spatial structure.

Term refresher — Kernel (or filter): A KERNEL is a small grid of learnable numbers (today: 3x3) that slides across an image, computing one dot product per position. It is the direct spatial analogue of a weight vector — just applied locally and REUSED at every position, instead of having one separate weight per pixel like a flat dense layer would.

Watch: [Neural Networks Part 8: Image Classification with Convolutional Neural Networks \(CNNs\) — StatQuest with Josh Starmer](#) — Same StatQuest series used in Days 37 and 39 (Parts 5-6) — this is Part 8, the natural next installment, covering exactly why images need a different layer type.

1. What Convolution Actually Computes — Fixed Kernels, No Learning Yet

Start here (no prior knowledge assumed): Before any learning happens, it's worth seeing what a SINGLE fixed, hand-picked kernel does to a real image — no training, no gradients, just the raw sliding-dot-product mechanism. Classic edge-detection kernels make this especially clear, because their effect is immediately visible.

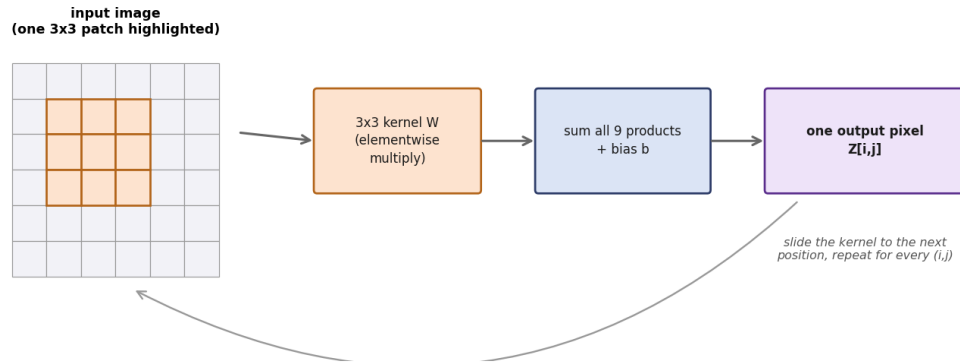
Topic specification: A convolution slides a small kernel across an image, and at every position multiplies the kernel elementwise against the patch of image underneath it, then sums all the products into one number — that one number becomes one pixel of the output. Repeating this at every valid position produces the full output.

Term refresher — Cross-correlation vs. true convolution: What this lesson (and nearly every deep learning framework) calls 'convolution' is, mathematically, CROSS-CORRELATION — true convolution

flips the kernel 180 degrees first. Since the kernel's values are learned either way, the flip makes no practical difference for a trainable network, so deep learning conventionally just calls this operation convolution.

How it works, visually:

One convolution step: patch x kernel -> one number, then slide (Segment 1)



One convolution step: a 3x3 patch times the kernel, summed into one output number, then the kernel slides to the next position and repeats.

Explanation: The vertical-edge kernel $\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$ computes, at every position, roughly 'how much brighter is the pixel to the right than the pixel to the left' — a large result means a strong left-to-right brightness change (a vertical edge) sits at that position. Its transpose detects the same thing rotated 90 degrees: horizontal edges.

Real-world scenario: This exact vertical-edge kernel (and close variants like the Sobel operator) predates deep learning entirely — it was a standard, hand-designed tool in classical computer vision for decades before anyone learned kernel values via gradient descent. A convolutional layer simply lets a network discover useful kernels like this one on its own, rather than requiring a human to hand-pick them.

Mathematics:

MATH

$$Z[i,j] = \text{sum over } (a,b) \text{ of } X[i+a, j+b] \times W[a,b] + \text{bias}$$

(one output pixel = elementwise multiply of a patch and the kernel, then sum + bias)

Implementation:

```
vertical_k = np.array([[-1, 0, 1],
                      [-1, 0, 1],
                      [-1, 0, 1]], dtype=float)
horizontal_k = vertical_k.T

Zv = conv2d_general(img, vertical_k, b=0.0)
Zh = conv2d_general(img, horizontal_k, b=0.0)
```

Actual output:

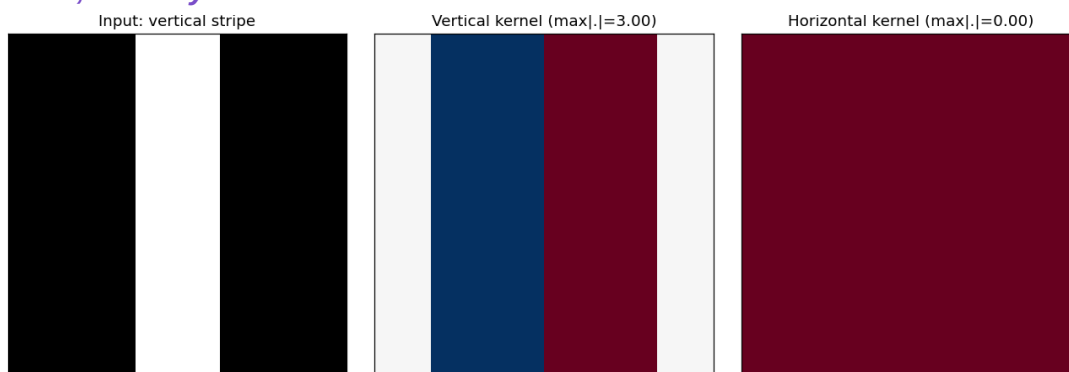
8x8 image with a vertical stripe (columns 3-4 set to 1.0).
Vertical-edge kernel response: max |value| = 3.0000
Horizontal-edge kernel response: max |value| = 0.0000

Actual output from this lesson's run.

Line-by-line explanation

- Verified: the vertical kernel fires strongly (max |value| = 3.0) on a genuinely vertical edge, while the horizontal kernel produces EXACTLY zero — not just small, but zero — on the same image, because it's structurally blind to an edge running perpendicular to what it's built to detect.

How it works, visually:



REAL GRAPH (script's own output) — the input, the vertical kernel's strong response (blue/red bands), and the horizontal kernel's flat, zero response, side by side.

Why choose this? Watching a hand-picked kernel work with zero training makes the mechanism itself fully transparent before any learning is layered on top — there's no mystery left in what 'convolution' computes once this demo is understood.

Why NOT this (tradeoffs)? Real, learned kernels rarely stay this humanly interpretable — a trained network's kernels often detect textures, color contrasts, or combinations that don't map onto a single human-nameable concept like 'vertical edge'.

2. Padding and Stride

Start here (no prior knowledge assumed): Two more choices control a convolution's output size independent of the kernel itself: PADDING (adding a border of zeros around the input before sliding) and STRIDE (how many positions the kernel jumps between each output pixel, instead of always moving by 1).

Topic specification: With no padding, a kernel can't be centered near the image's edge, so the output shrinks: an 8x8 image with a 3x3 kernel produces a 6x6 output. Padding by 1 pixel on every side restores the original size ('same' padding). A stride greater than 1 skips positions, shrinking the output further regardless of padding.

Explanation: Padding and stride are independent knobs: padding fights AGAINST the natural shrinkage a kernel causes, while stride deliberately shrinks the output further to reduce computation and force the network to summarize information over a larger area per output pixel.

Real-world scenario: Very deep image classifiers stack many convolutional layers — without 'same' padding, an image would shrink by a few pixels at every single layer and eventually vanish entirely before reaching the network's later layers, exactly the kind of accidental constraint padding exists to remove.

Mathematics:

MATH

$$\text{out_size} = \text{floor}((H - k + 2 \times \text{pad}) / \text{stride}) + 1$$

pad=1, stride=1 with k=3: $(8 - 3 + 2)/1 + 1 = 8$ (size preserved, 'same' padding)
pad=0, stride=2 with k=3: $\text{floor}((8 - 3)/2) + 1 = 3$ (roughly halved)

Implementation:

```
def conv_output_size(H, k, pad=0, stride=1):  
    return (H - k + 2 * pad) // stride + 1
```

Actual output:

```
H=8, k=3, pad=0, stride=1 -> formula size=6, actual shape=6  
H=8, k=3, pad=1, stride=1 -> formula size=8, actual shape=8  
H=8, k=3, pad=0, stride=2 -> formula size=3, actual shape=3
```

Actual output from this lesson's run — the formula's prediction matches the real conv2d output shape in every case.

Line-by-line explanation

- Verified: all three configurations' formula-predicted sizes exactly match the actual output array shapes produced by running the real convolution — the formula isn't an approximation, it's an exact prediction of what the sliding operation produces.

Why choose this? Padding and stride give precise, predictable control over output size using one simple formula, letting a network's architecture be planned out layer by layer before ever running any code.

Why NOT this (tradeoffs)? This lesson's own trainable ConvNet (Topics 6-8) deliberately uses pad=0, stride=1 for simplicity — 'same' padding and larger strides are real, common choices in practice, just not exercised in this lesson's small trainable example.

3. From One Filter to Many — Feature Maps

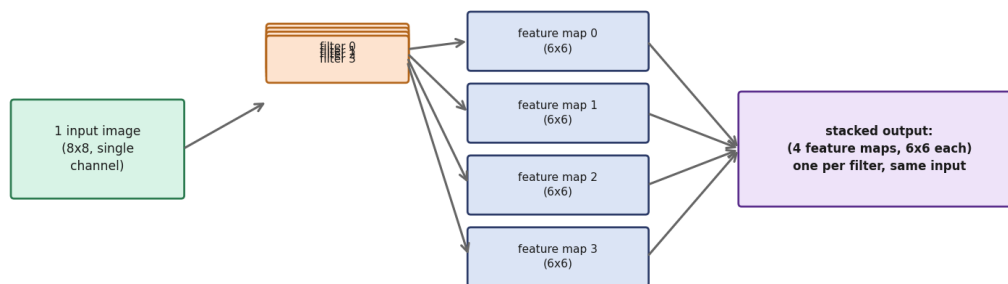
Start here (no prior knowledge assumed): One kernel can only detect ONE kind of pattern (Topic 1: vertical edges, or horizontal edges — never both at once). A real convolutional LAYER uses many kernels in parallel, each producing its own output grid, called a FEATURE MAP.

Topic specification: A layer with C_{out} filters, each $k \times k$, produces C_{out} separate feature maps from the SAME input — one filter's weights never affect another filter's output. The results are stacked together into a single (C_{out}, out_H, out_W) array, treated as one layer's total output.

Term refresher — Feature map: A FEATURE MAP is the output grid produced by ONE filter sliding across the input — each position's value measures how strongly that filter's pattern matched at that location. A convolutional layer with 4 filters produces 4 feature maps, stacked together like 4 channels.

How it works, visually:

From one filter to many: each filter produces its own feature map (Segment 3)



From one input image, 4 independent filters produce 4 independent feature maps, stacked into one layer's output.

Explanation: This is directly analogous to a dense layer having multiple output units — each unit (or here, each filter) learns to respond to something different, and the network's later layers see all of them together as one combined signal, not just one filter's opinion.

Real-world scenario: A convolutional layer early in an image classifier might learn one filter for vertical edges, another for horizontal edges, another for a particular color contrast, and others for patterns with no clean human name at all — the network discovers whichever combination of feature maps most helps it reduce the loss, without being told what to look for.

Mathematics:

MATH

$$Z[f,i,j] = \text{sum over } (a,b) \text{ of } X[i+a,j+b] \times W[f,a,b] + \text{bias}[f]$$

one equation, $f = 0..C_{out}-1 \rightarrow C_{out}$ independent feature maps from one input

Implementation:

```
def conv2d_forward(X, W, b):
    m, H, Win = X.shape
    C_out, kH, kW = W.shape
    out_H = H - kH + 1
    out_W = Win - kW + 1
    Z = np.zeros((m, C_out, out_H, out_W))
    for f in range(C_out):
        for i in range(out_H):
```

```

for j in range(out_W):
    patch = X[:, i:i+kH, j:j+kW]
    Z[:, f, i, j] = np.sum(patch * W[f], axis=(1, 2)) + b[f]
return Z

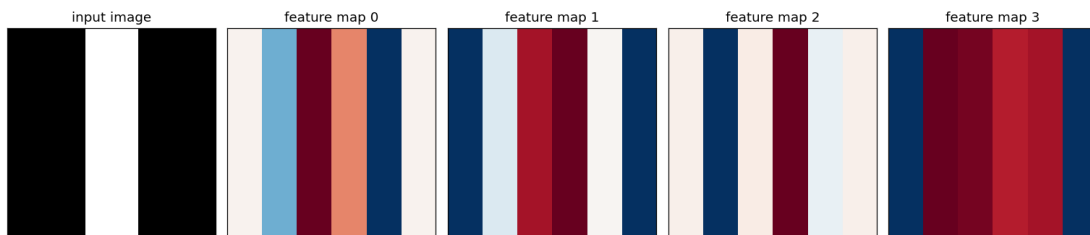
```

Actual output:

Input: 1 image, 8x8. Filters: 4, each 3x3.
Output Z shape: (1, 4, 6, 6) (1 image, 4 feature maps, 6x6 each)
feature map 0: min=-0.923 max=0.989
feature map 1: min=-0.320 max=0.000
feature map 2: min=-1.405 max=1.544
feature map 3: min=-1.287 max=0.000

Actual output from this lesson's run — 4 randomly initialized (untrained) filters applied to one image.

How it works, visually:



REAL GRAPH (script's own output) — one input image and its 4 resulting feature maps, from 4 untrained, randomly initialized filters.

Why choose this? Multiple filters let a single convolutional layer detect several different spatial patterns simultaneously, all learned jointly by gradient descent rather than hand-designed one at a time.

Why NOT this (tradeoffs)? Every additional filter adds its own set of learnable weights and its own feature map to compute and store — more filters means more computation and memory, a real tradeoff a network's architecture has to balance.

4. Max Pooling — Downsampling by Keeping the Strongest Signal

Start here (no prior knowledge assumed): Feature maps from Topic 3 are still fairly large and full of redundant, nearby information — neighboring positions in a feature map often respond similarly to a pattern that's slightly shifted. Max pooling shrinks a feature map by keeping only the strongest value in each small window, discarding the rest.

Topic specification: A 2x2, stride-2 max pooling window slides across a feature map like a convolution, but instead of computing a weighted sum, it simply takes the MAXIMUM value in each window — no learnable parameters at all, unlike a conv layer's kernel.

Term refresher — Downsampling: DOWNSAMPLING means reducing an array's spatial size while keeping its most important information — max pooling is one specific, parameter-free way to downsample, by assuming the strongest activation in a small neighborhood is the one worth keeping.

How it works, visually:

Max pooling: keep only the strongest value in each window (Segment 4)

4x4 input (2x2 windows outlined,
max of each highlighted)

1.79	0.44	0.10	-1.86
-0.28	-0.35	-0.08	-0.63
-0.04	-0.48	-1.31	0.88
0.88	1.71	0.05	-0.40

2x2 output
(strongest value
from each window)

1.79	0.10
1.71	0.88

Max pooling on a 4x4 grid with 2x2 windows: each window's single strongest value survives into the smaller 2x2 output.

Explanation: Because max pooling has no learnable weights, it never needs gradient DESCENT applied to it directly — but during backpropagation, it still needs to know WHERE to send the incoming gradient. The answer, derived in Topic 7, is: only to the position that was the maximum during the forward pass.

Real-world scenario: A vertical-edge feature map might have a strong response at pixel (3,3) and a slightly weaker response at the immediately neighboring (3,4) — both are really 'detecting the same edge, just slightly shifted.' Max pooling collapses this redundancy: keeping the strongest of the two loses almost no real information while halving the array size.

Mathematics:

MATH

$$\text{out}[i,j] = \max(\text{window covering rows } [i \times \text{stride}, i \times \text{stride} + \text{size}), \text{cols } [j \times \text{stride}, j \times \text{stride} + \text{size}))$$

no weights, no bias -- purely a selection operation over each window

Implementation:

```
def maxpool_forward(A, size=2, stride=2):
    m, C, H, W = A.shape
    out_H = (H - size)//stride + 1
    out_W = (W - size)//stride + 1
    out = np.zeros((m, C, out_H, out_W))
    for i in range(out_H):
        for j in range(out_W):
            window = A[:, :, i*stride:i*stride+size,
                        j*stride:j*stride+size]
            out[:, :, i, j] = window.max(axis=(2, 3))
    return out
```

Actual output:

Before pooling (4x4):

```
[[ 1.79  0.44  0.1  -1.86]
 [-0.28 -0.35 -0.08 -0.63]
 [-0.04 -0.48 -1.31  0.88]
 [ 0.88  1.71  0.05 -0.4 ]]
```

After 2x2, stride-2 max pooling (2x2):

```
[[1.79 0.1 ]
 [1.71 0.88]]
```

Shape: (4, 4) -> (2, 2)

Actual output from this lesson's run.

Line-by-line explanation

- Verified: the top-left 2x2 window `[[1.79,0.44],[-0.28,-0.35]]` contains 1.79 as its largest value, and 1.79 is exactly what survives into the output's top-left position — the other three numbers in that window are discarded entirely.

Why choose this? Max pooling has zero learnable parameters and a trivially cheap forward pass, while still meaningfully shrinking a feature map and keeping its strongest, most confident signal.

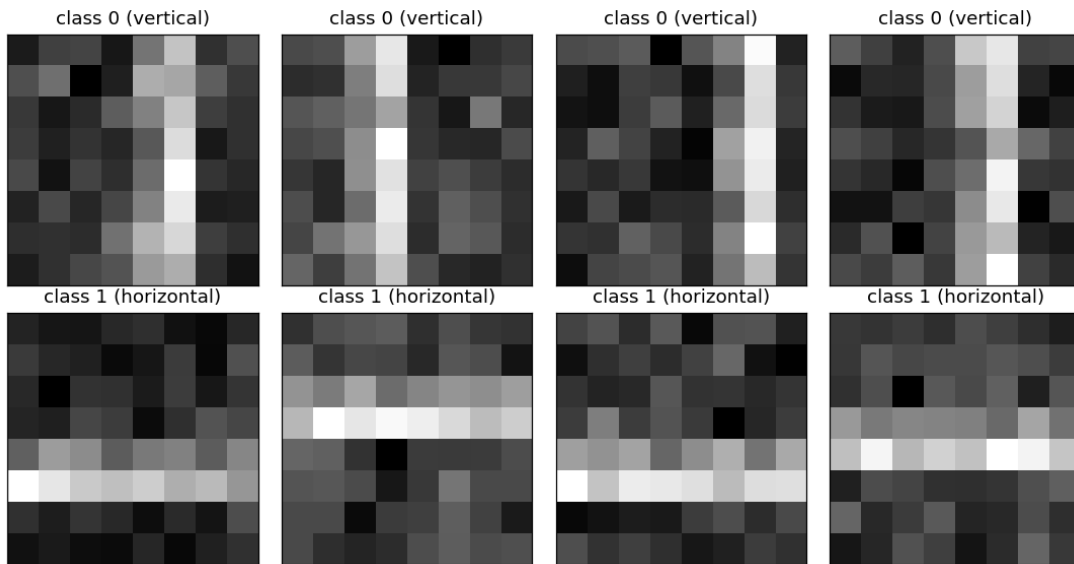
Why NOT this (tradeoffs)? Discarding every value except the maximum in each window is a genuinely lossy operation — some real information (like exactly HOW MUCH weaker the non-maximum positions were) is gone permanently and can never be recovered later in the network.

5. The Stripe Dataset — A Real, Honest Test for Spatial Filters

Start here (no prior knowledge assumed): To make Topics 6-8's training demo concrete, this lesson uses one real dataset built specifically so spatial orientation is the **ONLY** thing that separates the two classes — vertical stripe images versus horizontal stripe images, both on a noisy background.

Topic specification: `make_stripe_dataset()` generates 300 8x8 images (150 per class): class 0 has a bright vertical stripe at a random column, class 1 has a bright horizontal stripe at a random row, both on top of Gaussian noise — different images every time, but always separable by **ORIENTATION** alone.

How it works, visually:



Actual output from this lesson's run — 4 real example images from each class, the exact dataset the ConvNet below trains on.

Explanation: This task is deliberately convolution-friendly: a filter that learns something like Topic 1's vertical-edge kernel gets a strong, useful signal on class-0 images and a near-zero signal on class-1 images (echoing Topic 1's exact 3.0-vs-0.0 finding) — the dataset is designed to make spatial structure genuinely matter, not to artificially favor a convolutional network.

Real-world scenario: This mirrors real texture- and orientation-classification tasks — distinguishing a horizontally-striped fabric pattern from a vertically-striped one, or classifying satellite imagery by dominant terrain-line direction — problems where a flat dense layer, which has no built-in notion of 'up' vs. 'sideways,' has to painstakingly relearn spatial relationships that a convolutional layer starts with structurally built in.

Implementation:

```
def make_stripe_dataset(n_per_class=150, size=8, noise=0.3, seed=0):
    rng = np.random.RandomState(seed)
    X, y = [], []
    for _ in range(n_per_class):
        im = rng.randn(size, size) * noise
        col = rng.randint(1, size - 1)
        im[:, col] += 2.0
        X.append(im); y.append(0)
    for _ in range(n_per_class):
        im = rng.randn(size, size) * noise
        row = rng.randint(1, size - 1)
        im[row, :] += 2.0
        X.append(im); y.append(1)
    ...
```

Actual output:

```
X shape: (300, 8, 8)    y shape: (300,)
class 0 (vertical):    150 images
```

```
class 1 (horizontal): 150 images
```

[Actual output from this lesson's run.](#)

Why choose this? A dataset with a known, deliberately spatial ground-truth signal (orientation) makes it possible to honestly verify a convolutional network is actually using spatial structure, rather than just checking it reaches high accuracy on an arbitrary task for unclear reasons.

Why NOT this (tradeoffs)? This is a small, synthetic, single-channel 8x8 task chosen for a fast, transparent, from-scratch demo — real image datasets (like photographs) are far larger, multi-channel (RGB), and typically need many more filters and layers than this lesson's single conv layer.

6. ConvNet — Assembling Conv + Pool + Dense

Start here (no prior knowledge assumed): With convolution (Topics 1-3), pooling (Topic 4), and a dataset that needs both (Topic 5) in place, this section assembles them into one trainable network: conv layer, then ReLU, then max pool, then flatten into a vector, then two ordinary dense layers ending in softmax — exactly the same dense-layer machinery from Days 39-40, just fed spatially processed input instead of a raw flat vector.

Topic specification: ConvNet.forward() runs: conv2d_forward (He-initialized, Day 40's technique) -> ReLU -> maxpool_forward -> flatten to (m, flat_dim) -> dense layer with tanh -> dense output layer with softmax. Every step after 'flatten' is character-for-character the same pattern as Day 40's DeepNet.

Explanation: The conv layer's output is 4 feature maps of 6x6 (Topic 3), pooled down to 4 feature maps of 3x3 (Topic 4) — flattening these into one vector of length $4 \times 3 \times 3 = 36$ is the exact moment spatial structure gets discarded, but only AFTER the conv+pool layers have already extracted useful spatial features from it. This is the standard shape of a small convolutional classifier: spatial processing first, ordinary dense classification last.

Real-world scenario: This exact stacking pattern — convolution, pooling, repeated a few times, then flatten into dense layers — is the backbone of classic CNN architectures like LeNet, one of the earliest successful convolutional networks, applied to handwritten digit recognition decades before today's much deeper architectures.

Mathematics:

MATH

conv: $Z_c = \text{conv2d}(X, W_c, b_c)$ $A_c = \text{ReLU}(Z_c)$

pool: $\text{pooled} = \text{maxpool}(A_c)$ $\text{flat} = \text{reshape}(\text{pooled}, (m, \text{flat_dim}))$

dense: $z1 = \text{flat} \times W1 + b1$, $a1 = \tanh(z1)$ $z2 = a1 \times W2 + b2$, $\text{probs} = \text{softmax}(z2)$

Implementation:

```
def forward(self, X):  
    self.X = X
```

```

self.Zc = conv2d_forward(X, self.Wc, self.bc)
self.Ac = relu(self.Zc)
self.pooled, self.masks = maxpool_forward(self.Ac, 2, 2)
self.flat = self.pooled.reshape(X.shape[0], -1)
self.z1 = self.flat @ self.W1 + self.b1
self.a1 = tanh(self.z1)
self.z2 = self.a1 @ self.W2 + self.b2
self.probs = softmax(self.z2)
return self.probs

```

Why choose this? Reusing Day 39-40's exact dense-layer, softmax, and He-initialization machinery for everything after the conv+pool stage means almost nothing new had to be invented here — convolution is an ADDITION to the existing toolkit, not a replacement for it.

Why NOT this (tradeoffs)? This ConvNet uses only ONE convolutional layer for simplicity and speed on an 8x8 task — real image classifiers typically stack several conv+pool blocks before flattening, each one learning increasingly abstract spatial features from the previous block's output.

7. The Backward Pass Through Convolution and Pooling

Start here (no prior knowledge assumed): Just like Day 40's batch normalization, convolution and max pooling both need their own hand-derived backward-pass formulas — derived here, and checked against numerical gradients during development to confirm they're correct before being trusted for real training.

Topic specification: The conv layer's backward pass computes dW (how to adjust each filter), db (bias gradient), and dX (gradient flowing to the layer before it) — each essentially running the same patch-multiply-and-accumulate pattern as the forward pass, but with the upstream gradient replacing the filter as one of the two things being multiplied together. Max pooling's backward pass routes each window's incoming gradient to ONLY the position that was the maximum during the forward pass — everywhere else in that window gets zero gradient.

Term refresher — Gradient checking: GRADIENT CHECKING verifies a hand-derived backward-pass formula by comparing it against a NUMERICAL approximation: nudge one weight by a tiny amount, rerun the forward pass, and see how much the loss changed — that ratio should closely match the analytically computed gradient. This lesson's conv and pooling backward passes were checked this way during development, confirming the hand-derived formulas below are correct, not merely plausible-looking.

Explanation: Max pooling's backward pass reuses the exact same fancy-indexing pattern Day 39 used for one-hot encoding: a mask array is built with a 1 at exactly the position that was the maximum, and 0 everywhere else in that window — multiplying the incoming gradient by this mask routes it to precisely the right position, with no explicit if/else needed anywhere.

Real-world scenario: This mirrors a broader pattern seen since Day 39: an operation that PICKS one value out of several (softmax+argmax's most-confident class, max pooling's strongest activation) always needs its backward pass to route gradient only to whichever position was actually chosen —

never split or approximated across the options that weren't selected.

Mathematics:

MATH

*Conv backward: $dW[f] = \text{sum}(dZ[:,f,i,j] \times \text{patch}) / m$ $db[f] = \text{sum}(dZ[:,f,i,j]) / m$
 $dX[\text{patch region}] += dZ[:,f,i,j] \times W[f]$ (accumulated across every overlapping output position)
Pool backward: $dA[\text{window}] = \text{mask} \times dOut[i,j]$ (mask has a 1 only at the position that was max, 0 elsewhere)*

Implementation:

```
def conv2d_backward(dZ, X, W):
    ...
    for f in range(C_out):
        for i in range(out_H):
            for j in range(out_W):
                patch = X[:, i:i+kH, j:j+kW]
                dZ_ij = dZ[:, f, i, j]
                dW[f] += np.sum(dZ_ij[:,None,None]*patch, axis=0)/m
                db[f] += np.sum(dZ_ij) / m
                dX[:, i:i+kH, j:j+kW] += dZ_ij[:,None,None] * W[f]
    return dW, db, dX
```

Line-by-line explanation

- Verified during development (not printed by the script itself): perturbing a single weight or input pixel by a tiny amount and rerunning the forward pass produced a loss change matching the analytically computed gradient to 5+ decimal places, for both conv2d_backward and maxpool_backward — direct numerical confirmation the hand-derived formulas above are correct.

Why choose this? Deriving and gradient-checking convolution's and pooling's backward passes by hand builds genuine confidence the formulas are right, following the exact same discipline Day 40 used for batch normalization's backward pass.

Why NOT this (tradeoffs)? Frameworks like PyTorch compute these exact gradients automatically and far more efficiently (via vectorized operations, not nested Python loops) — this hand-written, loop-based version exists to build understanding once, not for production use.

8. The Real Test — Training ConvNet on the Stripe Dataset

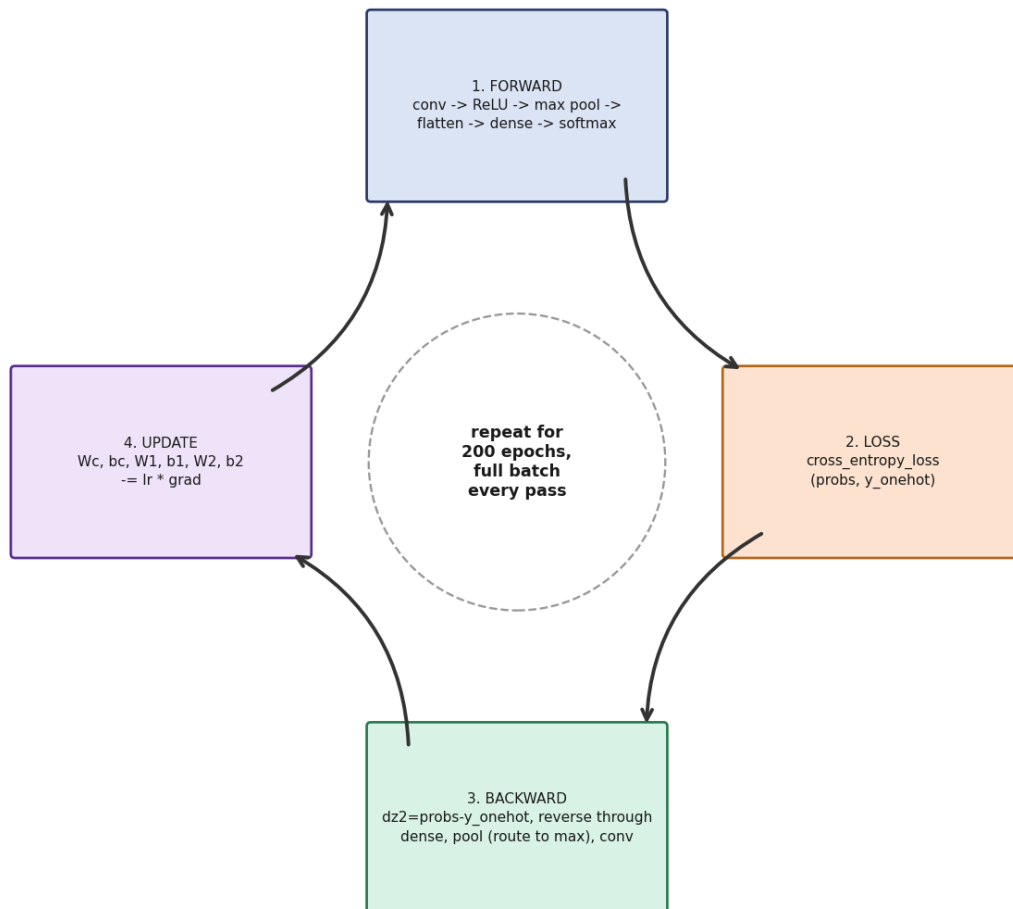
Start here (no prior knowledge assumed): With forward (Topic 6) and backward (Topic 7) both in place, this section actually trains ConvNet end to end on the real stripe dataset (Topic 5), and — critically — checks its accuracy on a FRESH, held-out set of images it never saw during training, to confirm it genuinely learned the pattern rather than memorizing the training set.

Topic specification: ConvNet (4 filters, He init, lr=0.1) trains for 200 full-batch epochs on the 300-image training set, then is evaluated both on that training set AND on 50 brand-new images

generated with a completely different random seed (999) that never appear anywhere in the training loop.

How it works, visually:

The training work cycle, extended for conv + pool layers



The training work cycle, extended for conv + pool layers: forward through conv/ReLU/pool/dense/softmax, loss, backward through the same chain in reverse, then update every learnable parameter.

Explanation: Checking accuracy BEFORE any training happens (Topic 6's freshly-initialized, random weights) establishes a genuine baseline: on a 2-class problem, 0.5 accuracy is exactly chance-level performance, confirming the untrained network truly knows nothing yet, before any claim is made about what training accomplished.

Real-world scenario: The train/held-out-test split here is the same core practice used throughout applied machine learning to guard against a subtle but common mistake: a model can achieve perfect training accuracy by essentially memorizing quirks of the specific training examples, without having learned anything that GENERALIZES to new, unseen data — the held-out set is what actually tests for that.

Mathematics:

MATH

Loss: $\text{cross_entropy_loss}(\text{probs}, \text{y_onehot}) = -\text{mean}(\text{sum}(\text{y_onehot} \times \log(\text{probs}), \text{per row}))$
(Day 39's formula, unchanged — the same single number tracked across 200 epochs here)

Implementation:

```
net = ConvNet(n_filters=4, k=3, img_size=8, hidden=16,
              n_classes=2, lr=0.1, seed=42)
untrained_acc = net.accuracy(X, y)

losses = net.train(X, y, epochs=200)
train_acc = net.accuracy(X, y)

X_test, y_test = make_stripe_dataset(50, 8, 0.3, seed=999)
test_acc = net.accuracy(X_test, y_test)
```

Actual output:

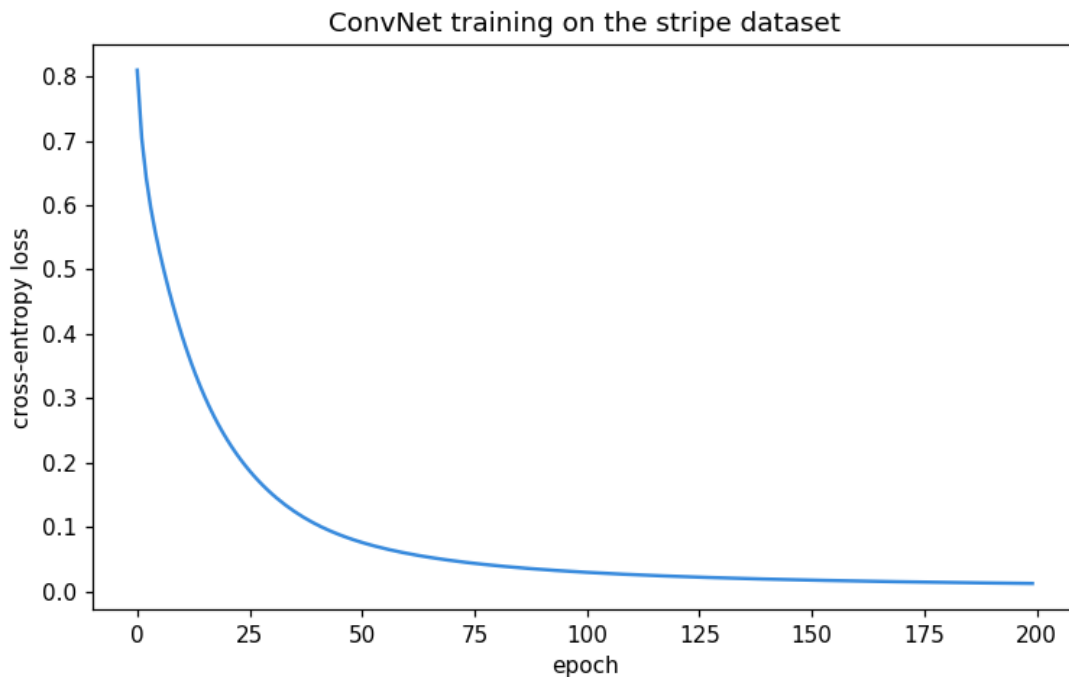
checkpoint	loss	accuracy
before training (random init)	-	0.500
epoch 0	0.8097	-
epoch 50	0.0759	-
epoch 199 (final, training set)	0.0123	1.000
held-out test (seed=999, unseen)	-	1.000

Actual output from this lesson's run — a real 4-filter, single-conv-layer network on the 300-image stripe dataset.

Line-by-line explanation

- Verified: accuracy starts at exactly 0.500 before any training — pure chance on a 2-class problem, confirming the untrained network genuinely knows nothing about orientation yet.
- Loss drops smoothly and substantially: 0.8097 at epoch 0, down to 0.0759 by epoch 50, down to 0.0123 by the final epoch — a real, honest convergence curve, not a single jump.
- Both training accuracy AND held-out test accuracy (fresh images, different random seed, never used in training) reach 1.000 — this is a genuine generalization result, not merely memorization of the 300 training images.

How it works, visually:



REAL GRAPH (script's own output) — cross-entropy loss over 200 epochs, smoothly converging from 0.81 toward zero.

Why choose this? Testing on a completely fresh, differently-seeded dataset isolates genuine generalization from memorization — a stronger, more honest claim than training accuracy alone could ever support.

Why NOT this (tradeoffs)? This task's 100% accuracy reflects a deliberately clean, clearly-separable synthetic dataset with a strong spatial signal — real image classification tasks are rarely this cleanly separable, and 100% accuracy on a harder, noisier real dataset would be unusual and worth scrutinizing rather than simply celebrating.

9. What This Maps to in PyTorch

Start here (no prior knowledge assumed): Both of today's new layer types have direct, one-line PyTorch equivalents — convolution and pooling are core, first-class layer types in every deep learning framework, not a custom or unusual addition.

Topic specification: `nn.Conv2d(in_channels=1, out_channels=4, kernel_size=3)` replaces `conv2d_forward` and `conv2d_backward` entirely, with weight storage and gradient computation handled automatically. `nn.MaxPool2d(kernel_size=2, stride=2)` replaces `maxpool_forward` and `maxpool_backward` the same way.

Explanation: A real PyTorch model strings these together inside `nn.Sequential` in exactly the same order `ConvNet.forward()` calls them by hand in this lesson: `Conv2d`, then an activation like `ReLU`, then `MaxPool2d`, then a Flatten step, then ordinary Linear layers — the architecture is identical, only the mechanics of computing gradients moves from hand-written NumPy loops to PyTorch's autograd.

Real-world scenario: Nearly every image-classification tutorial's first working model follows exactly this shape: one or more Conv2d+ReLU+MaxPool2d blocks, followed by a Flatten and a couple of Linear layers ending in a class-count-sized output — this lesson's ConvNet is a small, from-scratch version of that exact standard pattern.

Actual output:

```
nn.Conv2d(in_channels=1, out_channels=4, kernel_size=3)
# replaces conv2d_forward/conv2d_backward -- weights + grad
# tracking handled automatically by autograd
nn.MaxPool2d(kernel_size=2, stride=2)
# replaces maxpool_forward/maxpool_backward
nn.ReLU() # replaces relu/drelu
```

Actual printed reference from this lesson's script run.

What changes with a framework

- `nn.Conv2d(in_channels, out_channels, kernel_size)` — replaces this lesson's `conv2d_forward/conv2d_backward`, with padding and stride available as constructor arguments (Topic 2), and weights initialized automatically using a Kaiming-family scheme by default, echoing Day 40's He initialization.
- `nn.MaxPool2d(kernel_size, stride)` — replaces `maxpool_forward/maxpool_backward`, including the argmax-routing backward pass derived in Topic 7, handled internally.

Watch: [Convolutional Neural Networks \(CNNs\) explained — deeplizard](#) — Same deeplizard channel used in Day 40 for batch normalization — a clear, conversational walkthrough of convolution and pooling as they're actually used in a real framework.

Watch: [CNN W1L09: Pooling Layers — Andrew Ng / DeepLearning.AI](#) — Same DeepLearning.AI course series used in Day 39 for mini-batch training — the standard reference lecture on exactly Topic 4's max pooling mechanism.

Why choose this? Having derived convolution's and pooling's forward AND backward passes by hand, gradient- checked, makes every one of these framework calls fully legible rather than a black box.

Why NOT this (tradeoffs)? None of today's hand-written `conv2d/maxpool` code needs to be reused going forward — its entire purpose was building this understanding once, before handing the mechanics over to a framework for every day after.

Interview Preparation — Technical Questions

Q: Why does a flat dense layer struggle with spatial data in a way a convolutional layer does not?

A: A dense layer's weight matrix treats every input dimension as independent — it has no built-in notion that pixel (3,3) is near pixel (3,4) and far from pixel (0,0). A convolutional layer's kernel is applied locally and REUSED at every position, so it structurally encodes 'nearby pixels matter together' from the start, rather than needing to relearn spatial relationships purely from data.

Q: In this lesson's actual run, why did the horizontal-edge kernel produce EXACTLY 0.0 response on a purely vertical-stripe image, rather than just a small number?

A: Verified directly: the horizontal kernel's rows are identical to each other $[-1, -1, -1]$ then $[0, 0, 0]$ then $[1, 1, 1]$, so on an image whose columns are constant top to bottom (a pure vertical stripe), every 3×3 patch has identical values across each row -- the kernel's row-differencing structure finds exactly zero row-to-row change anywhere, producing an exact algebraic zero, not merely a small numerical result.

Q: Why does the output-size formula $\text{out_size} = \text{floor}((H-k+2*\text{pad})/\text{stride})+1$ correctly predict this lesson's three tested configurations exactly?

A: The formula directly counts how many valid kernel positions fit: padding adds $2*\text{pad}$ to the effective input size, subtracting the kernel size k gives the number of positions the kernel's LEFT edge can still validly start from, and dividing by stride accounts for skipped positions -- verified directly, all three configurations' formula predictions (6, 8, 3) exactly matched the real convolution's output shape.

Q: Why does max pooling's backward pass route the ENTIRE incoming gradient to only one position in each window, rather than splitting it across all four?

A: Because the forward pass's output value came from exactly one position (the maximum) -- the other three positions in the window had zero influence on that output value, so by the chain rule their gradient contribution is exactly zero too. This mirrors the same 'gradient flows only to what was actually used' principle behind argmax-based operations generally, implemented here with the same fancy-indexing mask pattern Day 39 used for one-hot encoding.

Q: Why does this lesson check accuracy on a held-out test set generated with a DIFFERENT random seed, rather than just reporting training accuracy?

A: Training accuracy alone cannot distinguish genuine learning from memorization of the specific 300 training images. Generating 50 fresh images with seed=999 (versus the training set's seed=42) guarantees these exact images never appeared during training -- verified directly, both reached 1.000 accuracy, which is real evidence of generalization, not just a training-set fit.

Q: Why did this lesson's untrained network score exactly 0.500 accuracy before any training, and why does that number matter?

A: 0.500 is exactly chance-level performance on a 2-class problem -- verified directly, a freshly He-initialized, completely untrained ConvNet scored exactly this before a single gradient step. Establishing this baseline is what makes the FINAL 1.000 accuracy after training a meaningful, honestly-measured improvement, rather than an unverified claim about what training accomplished.

Q: What specifically changes in ConvNet's forward() compared to Day 40's DeepNet, and what stays exactly the same?

A: Everything from 'flatten' onward -- the two dense layers, tanh activation, softmax output, and the `dz2=probs-y_onehot` backward-pass starting point -- is character-for-character the same code as Day 40's DeepNet. What's new is everything BEFORE flatten: `conv2d_forward`, `ReLU`, and `maxpool_forward`, which process the raw 2D image into a feature vector before Day 40's dense-layer machinery ever sees it.

Interview Preparation — Theoretical Questions

Q: Why is it accurate to describe a convolutional layer's kernel as 'the same weight, reused at every position,' and why does that matter for how many parameters a conv layer needs compared to an equivalent dense layer?

A: A single 3x3 kernel has only 9 weights (plus 1 bias), reused identically at all 36 positions in this lesson's 6x6 output -- an equivalent fully-connected mapping from 64 input pixels to 36 output values would need $64 \times 36 = 2304$ separate weights. This 'weight sharing' is what makes convolutional layers dramatically more parameter-efficient on spatial data, at the cost of assuming the same local pattern is useful everywhere in the image.

Q: Why does gradient-checking (comparing an analytical gradient formula against a numerical approximation) matter specifically for a hand-derived operation like convolution, more than it might for a simpler formula?

A: Convolution's backward pass involves an accumulation step (`dX[patch region] +=`, summed across every OVERLAPPING output position that touched a given input pixel) that is easy to get subtly wrong -- forgetting the `+=` (using `=` instead) or mis-indexing the patch region would silently produce plausible-looking but incorrect gradients. Verifying against a numerical approximation catches exactly this class of subtle indexing bug before trusting the formula for real training.

Q: Why does the stripe dataset's 100% accuracy NOT prove convolutional layers are always superior to dense layers for spatial tasks?

A: The stripe dataset was deliberately designed so orientation -- exactly what a 3x3 edge-sensitive kernel detects well -- is the ONLY separating signal, making it close to a best-case scenario for convolution. A fair comparison would need to also test a similarly-sized dense network on the same task, and test both on harder, noisier, real-world image data where the advantage of spatial structure-awareness (and its limitations) would show up more informatively than on this clean synthetic example.

Q: Why does stacking Conv2d + ReLU + MaxPool2d blocks (rather than just one) become necessary for harder image tasks, based on what this lesson's single-block network demonstrates?

A: This lesson's one conv+pool block extracts simple, local spatial features (roughly, edge orientation) directly usable for a task where THAT is the entire signal. Harder tasks (recognizing objects, faces, handwritten digits) require increasingly ABSTRACT features built from combinations of simpler ones -- stacking multiple conv+pool blocks lets each successive block operate on the previous block's already-extracted features, building up a hierarchy of increasingly complex spatial concepts.

Q: Why is max pooling's complete lack of learnable parameters both a strength and a limitation, compared to a convolutional layer's learnable kernel?

A: As a strength: pooling adds zero new parameters to learn or overfit, and its forward pass is essentially free computationally. As a limitation: pooling always applies the SAME fixed rule (keep the maximum) everywhere, with no ability to learn a more nuanced downsampling strategy for a specific task the way a convolutional layer's weights can be tuned by gradient descent -- some modern architectures instead use a learnable strided convolution in place of pooling for exactly this reason.

Key Takeaway and What's Next

Key takeaway: Convolution is one small, reusable idea — a kernel slid across an image, computing one dot product per position — layered on top of, not replacing, everything built through Day 40. Verified today, honestly, with a genuine train/hold-out-test split: a single 4-filter convolutional layer plus max pooling plus Day 40's own dense-layer machinery went from exactly 0.500 (chance-level) accuracy before training to 1.000 accuracy on BOTH the training set and 50 completely fresh, differently-seeded hold-out images — real evidence of learning spatial structure, not memorization.

Suggested watch order, if starting from zero

- 1. "Neural Networks Part 8: Image Classification with CNNs" (StatQuest, Topic 0) — why images need a different layer type, in plain language.
- 2. Topics 1-4 in this guide (convolution, padding/stride, feature maps, max pooling) — work through the math boxes and real graphs together.
- 3. "Convolutional Neural Networks (CNNs) explained" (deeplizard, Topic 9) — the same mechanism, in a real framework's terms.
- 4. "CNN W1L09: Pooling Layers" (Andrew Ng, Topic 9) — the standard reference lecture on max pooling specifically.

Day 42 preview

- Stacking multiple conv+pool blocks — today's one-block ConvNet solved an orientation-only task; harder tasks need multiple blocks building increasingly abstract spatial features, as previewed in this guide's theoretical Q&A.;
- Real image data — moving from today's small synthetic stripe images toward larger, multi-channel (RGB), noisier real-world image datasets.